

CMSC451 Algorithms Notes

JAMES ZHANG^{*}

September 15, 2024

These are my notes for UMD's CMSC 451: *Design and Analysis of Algorithms*. They are taken live during class. This course is taught by Dr. Andrew Childs.

Contents

1	August 27, 2024	2
1.1	Algorithms	2
1.2	Stable Matching Problem	3
2	August 29, 2024	3
2.1	Gale-Shapely Algorithm	4
3	September 3, 2024	5
3.1	Asymptotic Notation	5
3.2	Analyzing Running Times	6
3.3	Graphs	7
4	September 5, 2024	7
4.1	Graph Representations	7
4.2	Paths and Cycles	8
4.3	Connectivity	8
4.4	Trees	8
5	September 10, 2024	9
5.1	Graph Algorithms	9
5.1.1	Breadth-First Search	9
5.2	Bipartiteness	10
6	September 12, 2024	11
6.1	Connectivity in Digraphs	11
6.2	Topological Sorting	11
7	September 17, 2024	13

^{*}Email: jzhang72@terpmail.umd.edu

§1 August 27, 2024

§1.1 Algorithms

Definition 1.1. An **algorithm** is a well-defined computational procedure for processing some input to obtain a desired output.

This course will discuss how to 1. design algorithms and 2. analyze their performance. Why study algorithms? There are several reasons:

- Coding interviews
- They arise in software; may want to make software efficient even on large inputs
- Solve a “one-off” problem
- Algorithm ideas are useful in other areas of science/technology
- Interesting math

This course will be structured in four parts:

1. Introduction

- Example: stable matching problem
- Mathematical background
- Basic graph algorithms

2. Algorithm design techniques

- Greedy algorithms
- Divide-and-conquer

3. Intractability

- NP-Completeness
- Approximation algorithms

4. Other models of computation

- Randomized algorithms
- Quantum algorithms

Algorithms are mathematical objects. Specifying an algorithm requires 1) pseudocode and 2) a plain language description (e.g. English).

Example 1.2

The following are good descriptions of algorithms:

- Let `foo` be a counter initialized to 0. For every even value of i from 0 to 100, if the i th entry of x is nonzero, increment `foo`
- Repeatedly find adjacent elements that are out of order and swap them until the list is sorted

We now go over a basic problem: the **Stable matching problem**.

§1.2 Stable Matching Problem

The goal of the stable matching problem is to pair members of two sets according to their preferences. For example, students and employers for summer internships or men and women to be married. For simplicity, the two sets have the same size. We must match each member of the first set with exactly one member of the second set.

Definition 1.3. Given two sets S, T , a **matching** is a set of pairs of the form (s, t) , $s \in S, t \in T$ such that every element of S appears in at most one pair; and similarly for T . A matching is **perfect** if every element of S and T is in some pair.

In the stable matching problem, each member ranks all elements of the other set (no ties; no omissions).

Definition 1.4. A matching is **stable** if there is no pair (s, t) **not** in the matching such that s prefers t over its assigned partner and t prefers s over its assigned partner.

§2 August 29, 2024

We continue with an example of the stable matching problem.

Example 2.1

Suppose we want to match students A, B to companies X, Y . The preferences of each are as follows:

A: X, Y X: A, B
B: Y, X Y: B, A

These preferences give the following stable matchings: (A, X) and (B, Y) .

Now, consider the below example:

Example 2.2

A: X, Y X: A, B
B: X, Y Y: A, B

These preferences give the following stable matching: (A, X) and (B, Y) . If we have the matching (B, X) and (A, Y) (A, X) is an instability.

Consider now the below example with three students and three companies:

Example 2.3

A: X, Y, Z X: B, C, A
 B: X, Z, Y Y: C, A, B
 C: Y, Z, X Z: A, C, B

Is the following matching stable? $(A, X), (B, Z), (C, Y)$. No, because (B, X) would be better together.

We now discuss the **Gale-Shapely algorithm** to find stable matches.

§2.1 Gale-Shapely Algorithm

The Gale-Shapely algorithm solves the stable matching problem as follows: suppose that first, all companies and students are unengaged. While there is a company c that is unengaged and has not made an offer to every student, let s be the highest-ranked student for c to whom c has not yet made an offer. If s is unengaged, (c, s) become engaged. Else, let c' be the company to which s is engaged. If s prefers c to c' , then (c, s) becomes engaged and c' becomes unengaged. The set of engaged pairs can be returned after the while loop.

Lemma 2.4

Once a student receives an offer, they remain engaged for the rest of the algorithm and the quality of their engagement (according to their preferences) can only increase.

Proof. An unengaged student who receives an offer always becomes engaged. An engaged student can only switch engagements; they can never become unengaged. A student only switches if they prefer the new company. $\square$

Lemma 2.5

As the algorithm proceeds, the sequence of students engaged to a given company can only decrease in preference.

Proof. Companies make offers to students in order of decreasing preference. $\square$

Lemma 2.6

If there are n companies and n students, then the algorithm terminates after at most n^2 iterations of the while loop.

Proof. Let $P(t)$ be the set of pairs (c, s) such that c has made an offer to s by the t th iteration. There is a new offer in every iteration, so $|P(t+1)| = |P(t)| + 1$. Since there are n^2 possible pairs, $|P(0)| = 0$ and $|P(t)|$ increases by 1 at each step, there can be at most n^2 steps. $\square$

§3 September 3, 2024

Today, we will continue discussing stable matching. Assignment #1 is available on Canvas and is due on September 18. Also, it is recommended to review the mathematical background for this class: asymptote notation, running time analysis, and graphs.

Lemma 3.1

The Gale-Shapely algorithm outputs a perfect matching.

Proof. New engagements are only created between parties that were not previously engaged or who broke a previous engagement to create a new one. Thus the set of engaged pairs is always a matching.

Suppose that at the end of the algorithm (which occurs by Lemma 2.6), there is some company c and student s who are not engaged. At some point, c must have made an offer to s (by the termination condition of the loop). By Lemma 1, s will be engaged at the end of the algorithm, a contradiction. $\square$

Theorem 3.2

The matching output by the Gale-Shapely algorithm is stable.

Proof. Suppose there is some pair (c, s) such that c prefers s to its intern s' and s prefers c to their company c' . Then c must have made an offer to s before s' . But c ended up with s' . What went wrong? Either:

- s had an offer they preferred to s , so they rejected the offer
- s accepted the offer from c but later switched to an even better company

But s ends up with c' , which is lower on their preference list than c . This is a contradiction. $\square$

We will now discuss **asymptotic notation**.

§3.1 Asymptotic Notation

We're interested in asymptotic upper bounds on the worst-case running times of algorithms. We can express this using big-O notation.

Example 3.3

The running time of $10n^2 + 2n + 5000$ is $\mathcal{O}(n^2)$.

Definition 3.4. We say $f(n) \in \mathcal{O}(g(n))$ if $\exists c > 0, n_0 \geq 0$ such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$.

We will think of algorithms as efficient if they run in true polynomial in the input size n , i.e. $\mathcal{O}(n^a)$ for some constant a .

Definition 3.5. We say $f(n) \in \Omega(g(n))$ if $\exists c > 0, n_0 \geq 0$ such that for all $n \geq n_0$, $f(n) \geq c \cdot g(n)$.

Example 3.6

$10n^2 \in \mathcal{O}(n^2), \in \mathcal{O}(n^3), \in \Omega(n^2), \Omega(n)$.

Definition 3.7. We say $f(n) \in \Theta(g(n))$ if $f(n) \in \mathcal{O}(g(n))$ and $f(n) \in \Omega(g(n))$.

Example 3.8

$10n^2 \in \Theta(n^2)$.

Definition 3.9. We say $f(n) \in o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$. We say $f(n) \in \omega(g(n))$ if $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$.

Example 3.10

$n^2 \in o(n^3), n^2 \in \mathcal{O}(n^3)$.

§3.2 Analyzing Running Times

We can analyze runtimes of algorithms in the following ways:

- Computing sums (for iterative algorithms)
- Solving recurrences (recursive algorithms)

Some common running times of algorithms are as follows:

- Logarithmic time: $\mathcal{O}(\log n)$
- Linear time: $\mathcal{O}(n)$
- Time $\mathcal{O}(n^k)$ for some constant k (for example, $\binom{n}{k} \in \mathcal{O}(n^k)$)
- Exponential time: $\mathcal{O}(2^n)$

We will now start discussing **graphs**.

§3.3 Graphs

Definition 3.11. An **undirected** graph $G = (V, E)$ is a finite set V of **vertices** and E of **edges**, where edges are unordered pairs of vertices.

Definition 3.12. A **directed** graph $G = (V, E)$ is a finite set of **vertices** and E of **edges**, where edges are ordered pairs of vertices.

§4 September 5, 2024

Today, we will discuss some math relating to graphs, as well as graph algorithms (connectivity, BFS, bipartiteness). We start by discussing terminology relating to graphs:

- **Vertices** are sometimes called **nodes**
- An edge $\{u, v\}$ **joins** u and v , which are its **ends**; u and v are **adjacent** if there is an edge joining them; the edge is **incident** on u and v
- An edge $\{u, v\}$ has **tail** u and **head** v
- The **degree** of vertex v , $\deg(v)$ is the number of incident edges it has
- In a digraph, $\text{indeg}(v)$ is the number of edges with v as a head and $\text{outdeg}(v)$ is the number of edges with v as a tail
- We often write $|V| = n$, $|E| = m$
- $\sum_{v \in V} \deg(v) = 2m$ and $\sum_{v \in V} \text{indeg}(v) = m = \sum_{v \in V} \text{outdeg}(v)$
- $m \leq \binom{n}{2} = \mathcal{O}(n^2)$ in an undirected graph. $m \leq n^2$ in a directed graph
- A **subgraph** of $G = (V, E)$ is a graph $G' = (V', E')$ with $V' \subseteq V$ and $E' \subseteq E$

We now discuss graph representations.

§4.1 Graph Representations

There are two main ways to represent graphs:

- **Adjacency matrix:** $|V| \cdot |V|$ matrix indexed by vertices with $A_{uv} = \begin{cases} 1 & \text{if } (u, v) \in E \\ 0 & \text{otherwise} \end{cases}$
- **Adjacency list:** for each $v \in V$, given a list Adj_v of its v neighbors outgoing

§4.2 Paths and Cycles

Definition 4.1. A **path** is a sequence of vertices $(v_0, v_1, v_2, \dots, v_k)$ such that $(v_i, v_{i+1}) \in E$ for all $i \in \{0, 1, \dots, k-1\}$.

Definition 4.2. A path is **simple** if all its vertices and edges are distinct. The **length** of a path is k , its number of edges.

Definition 4.3. A **cycle** is a path with $v_0 = v_k$ and all other vertices distinct. The **distance** between $u, v \in V$ is the length of a shortest path from u to v .

§4.3 Connectivity

Definition 4.4. An undirected graph is **connected** if for all $u, v \in V$, there exists a path from u to v .

Definition 4.5. A **component** of a graph is a set of vertices such that all pairs of vertices in the set have a path between them and no superset has this property.

Remark 4.6. We use the term **digraph** as a short form of directed graph.

Definition 4.7. A digraph is **strongly connected** if $\forall u, v \in V$, there exists a path from u to v and a path from v to u . A **strongly connected component** is a maximal subgraph that is strongly connected.

We now briefly discuss **trees**.

§4.4 Trees

Definition 4.8. A **forest** is a graph with no cycles.

Definition 4.9. A **tree** is a connected forest. A n -vertex tree has $n - 1$ edges. If we designed one vertex of a tree as the **root**, then the (unique) neighbor of a vertex that is closer to the root is its **parent** and the neighbors that are further from the root are its **children**. Vertices of degree 1 in a tree are called **leaves**.

Definition 4.10. A digraph with no cycles is called a **directed acyclic graph**, DAG for short.

An important question about connectivity is as follows: given a graph, is it connected? Given vertices s, t , is there a path between them?

One approach to solving this is to find a **spanning tree**, a subgraph that includes all vertices and is a tree. We can specify a rooted tree by giving the parent $\text{pr}[v]$ of every vertex v , with $\text{pr}[\text{root}] \neq \emptyset$. We now present a generic spanning tree algorithm:

Let T be the graph with one vertex s , with $\text{pr}[s] \neq \emptyset$

While $\exists$ an edge $\{u, v\}$ joining a vertex u of T to a vertex v not in T

 Add vertex v and edge $\{u, v\}$ to T

 Set $\text{pr}[v] = u$

The resulting graph is always a tree.

Lemma 4.11

The above algorithm produces a spanning tree for the component of s .

Proof. Any vertex in the tree has a path to s : repeatedly take parents until you reach s . To find a path from u to v in T , join the paths from u to s and v to s . On the other hand, if u is not in T , then there is no path to s . If there were, we could follow the path and find an edge from a vertex not in T for a vertex in T , but no such edge remains when the algorithm is done. $\square$

§5 September 10, 2024

We will now discuss some graph algorithms.

- Breadth-first search
- Bipartiteness
- Connectivity in directed graphs
- Topological sorting

§5.1 Graph Algorithms

§5.1.1 Breadth-First Search

Breadth-first search explores a graph in order of increasing distance from the root. Say a vertex is **exhausted** if it is not adjacent to any vertex outside the tree. The BFS pseudocode is as follows:

BFS(s):

Let T be the tree with one vertex, s , with $\text{pr}[s] \neq \emptyset$

Repeat until all vertices are exhausted

Let u be the unexhausted vertex that joined the tree earliest (the “active vertex”)

Choose an edge $\{u, v\}$ where v is not in T

Add vertex v and edge $\{u, v\}$ to T

Set $\text{pr}[v] = u$

We now move on to the problem of computing distances/shortest paths. BFS partitions the vertices into **layers** L_j , where L_0 is the starting vertex $\{s\}$ and L_{j+1} is the set of neighbors of vertices in L_j that are not also neighbors of vertices in $L_0 \cup L_1 \cup \dots \cup L_{j-1}$.

Lemma 5.1

A vertex in L_j has distance j from s .

Proof. We proceed by induction on j . Clearly $L_0 = \{s\}$ is the set of vertices at distance 0 from s . If $L_0, L_1, \dots, L_j$ satisfy the lemma, then the vertices in L_{j+1} must have distance at least $j+1$ from s . But there exists a path of length $j+1$ from any vertex in L_{j+1} to s since there is an edge to a vertex in L_j , which has a path of length j to s (by the inductive hypothesis). Hence vertices in L_{j+1} satisfy the lemma, so the lemma follows by induction. $\square$

What is the running time of BFS (in terms of $n = |V|$, $m = |E|$)? The algorithm is looping over all $u \in V$ and for each, it does a $\mathcal{O}(\deg(u) + 1)$ operation for each. The total running time is therefore $\mathcal{O}(\sum_{u \in V} (\deg(u) + 1)) = \mathcal{O}(m + n)$.

We now discuss bipartiteness in a graph.

§5.2 Bipartiteness

Lemma 5.2

Let T be a BFS tree of graph G and let u, v be vertices of T with $u \in L_i$ and $v \in L_j$. Suppose $\{u, v\}$ is an edge of G . Then $|i - j| \leq 1$.

Proof. WLOG suppose u is added before v . Then $i \leq j$ and u is active before v . We now have two cases:

1. v is in the tree when u becomes active. Then $\{u, v\}$ is a non-tree edge and $\text{pr}(v)$ joined the tree before u , so $j = \text{layer}(\text{pr}(v)) + 1 \leq i + 1$.
2. v is not in the tree when u becomes active. Then $\{u, v\}$ is a tree edge and $j = i + 1$.

$\square$

Definition 5.3. A graph $G = (V, E)$ is **bipartite** if there is a partition (A, B) of V ($V = A \cup B$, $A \cap B = \emptyset$) such that for all $\{u, v\} \in E$, either $u \in A$ and $v \in B$ or $u \in B$ and $v \in A$.

Theorem 5.4

A connected graph with BFS tree T is bipartite iff there is no non-tree edge joining vertices in the same layer.

Proof. Suppose there is no non-tree edge joining vertices in the same layer. Let $A = L_0 \cup L_2 \cup \dots \cup L_{2k}$ (vertices in even layers) and $B = L_1 \cup L_3 \cup \dots \cup L_{2k+1}$ (vertices in odd layers). This is a bipartition since all non-tree edges join vertices whose layers differ by 1.

Conversely, suppose $\{u, v\}$ is a non-tree edge between vertices in the same layer. Suppose their nearest common ancestor is j layers higher. Then the path in the tree between u and v has length $2j$. Adding $\{u, v\}$ gives a cycle of length $2j + 1$. But a graph with an odd cycle cannot be bipartite. $\square$

§6 September 12, 2024

Today, we will discuss more graph algorithms, specifically connectivity in directed graphs and topological sorting. We will also start discussing greedy algorithms.

§6.1 Connectivity in Digraphs

How can we test if a graph is strongly connected? Run BFS from every vertex (n runs of BFS); the digraph is strongly connected iff all runs reach all vertices. This is done in $\mathcal{O}((m+n)n)$ time.

Lemma 6.1

If u and v are mutually reachable and v and w are mutually reachable, then u and w are mutually reachable.

Proof. Note that we can go from $u \rightarrow w$ by going from $u \rightarrow v \rightarrow w$ and that we can go from $w \rightarrow u$ by going from $w \rightarrow v \rightarrow u$. $\square$

An algorithm to test if a graph is strongly connected is as follows:

- Run BFS starting from s
- Run BFS starting from s , reversing the directions of all edges
- If both searches find all vertices, the graph is strongly connected; otherwise not

The above runs in $\mathcal{O}(m+n)$ time.

§6.2 Topological Sorting

We can use a digraph to capture a set of operations with constraints on which ones must be done before others.

Definition 6.2. A **topological ordering** is a possible sequence in which the actions can be performed, subject to the constraints on which other actions must have been completed.

Lemma 6.3

If G has a topological sorting, then G is a DAG.

Proof. We will proceed by contradiction. Suppose G has a topological ordering and a directed cycle C $v_1, v_2, \dots, v_n$. Let v_i be the lowest-index vertex on C ; let v_j be the vertex before it on C . Then (v_j, v_i) is an edge with $i < j$, but we also must have $j < i$ for this to be a topological ordering. This is a contradiction. $\square$

Lemma 6.4

Every DAG has a vertex with indegree 0.

Proof. We will prove the contrapositive: if every vertex has positive indegree then the graph has a directed cycle.

Start at any vertex. Walk along some edge in the backwards direction (which is possible because every vertex has positive indegree). Repeat. After at most $n + 1$ steps, we have visited some vertex twice, so we found a cycle. $\square$

Lemma 6.5

Every DAG has a topological ordering.

Proof. We proceed by induction on $n = |V|$: the base case is $n = 1$; a 1-vertex graph trivially has a topological ordering.

The inductive hypothesis is as follows: suppose the lemma holds for any digraph with at most n vertices. For an $(n + 1)$ -vertex graph, let v be a vertex with indegree 0. Put it first in the ordering. Follow it by a topological ordering of $G \setminus \{v\}$, which is a graph with n -vertices (note that deletion of a vertex and edge from a graph cannot create a cycle, so this graph is still a DAG). $\square$

Theorem 6.6

A graph is a DAG if and only if it has a topological ordering.

We now present pseudocode for topological sort:

TopoSort(G):

```

Let  $S$  be an empty set
For all vertices  $v$ 
  If  $v$  has indegree 0, add  $v$  to  $S$ 
  Set  $\text{count}[v] = \text{indeg}(v)$ 
While  $S$  is nonempty do
  Remove a vertex  $v$  from  $S$ 
  Output  $v$ 
  For each vertex  $u$  that  $v$  points to
    Remove edge  $(v, u)$  from the graph
    Decrement  $\text{count}[u]$ 
    If  $\text{count}[u] = 0$  then add  $u$  to  $S$ 
```

To analyze the runtime of this algorithm, note that the algorithm runs in $\mathcal{O}(n)$ time for initialization; the while loop goes through all vertices, and the for loop for v takes

$\mathcal{O}(\text{outdeg}(v) + 1)$ steps, giving a total runtime of $\mathcal{O}(n + \sum_{v \in V} (\text{outdeg}(v) + 1)) = \mathcal{O}(m + n)$ time.

§7 September 17, 2024